CMSC132: Heaps

By Naren Ayinala

May 2026

Contents

1 What Is a Heap? 2
1.1 The Complete Tree Property 2

2 Array Representation of a Binary Heap 2
2.1 Example: Finding Relatives 2

3 Min Heaps and Max Heaps 3
3.1 Min Heap 3
3.2 Max Heap 3

4 Heap Operations 3
4.1 Insertion and Bubble Up 3
4.2 Removing the Root and Bubble Down 4
4.3 Peek 4

5 Building a Heap from an Unordered Array 4

6 Complete Java Min Heap Implementation 5

7 PriorityQueue in Java 6
7.1 Min Heap with PriorityQueue 6
7.2 Max Heap with PriorityQueue 7
7.3 Priority Queues with Custom Objects 7

8 Heaps vs Binary Search Trees 7

9 Time Complexity Summary 8

10 Heap Sort 8

11 Worked Practice Questions 8

12 Common Mistakes to Avoid 8

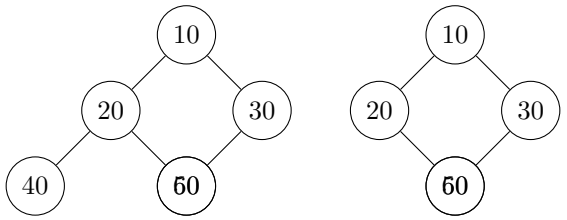
1 What Is a Heap?

A **binary heap** is a binary tree with two rules: (1) it must be a **complete binary tree**, and (2) it must satisfy the **heap-order property**. A complete binary tree has every level filled except possibly the last, which is filled from left to right with no gaps. This is why a binary heap can be represented using an array rather than nodes with explicit left and right references.

Remember: A heap is *not* the same thing as Java's heap memory. Heap memory stores objects; a binary heap is a data structure used to efficiently find and remove the minimum or maximum priority.

1.1 The Complete Tree Property

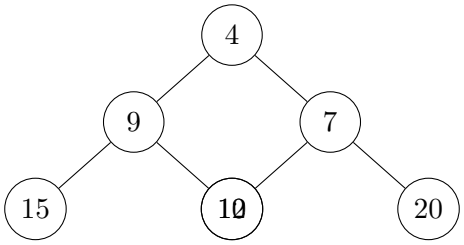
Consider the following two trees. The first is complete because the bottom level is filled left to right. The second is not complete because a gap appears before a later node.



Complete Not complete

2 Array Representation of a Binary Heap

Because a heap is complete, we can place its nodes in an array in level order. The root is stored at index 0, followed by the next level from left to right, and so on. We can calculate parent and child positions using index arithmetic instead of storing pointers.



Value	4	9	7	15	12	10	20
Index	0	1	2	3	4	5	6

For a node at index i in a **zero-indexed array**, we have:

$$\boxed{\text{left child} = 2i + 1} \qquad \boxed{\text{right child} = 2i + 2} \qquad \boxed{\text{parent} = \left\lfloor \frac{i-1}{2} \right\rfloor}$$

The parent formula matters: in your original notes, the parent was written as $\lfloor i/2 \rfloor$, but that is the *one-indexed* convention. With a zero-indexed Java array, we must use $\lfloor (i-1)/2 \rfloor$ for $i > 0$.

2.1 Example: Finding Relatives

Suppose $i = 4$. Its parent is $\lfloor (4-1)/2 \rfloor = 1$, so the node with value 12 has parent 9. Its left child index would be 9 and its right child index would be 10, but those positions are outside our seven-element array. So it has no children.

3 Min Heaps and Max Heaps

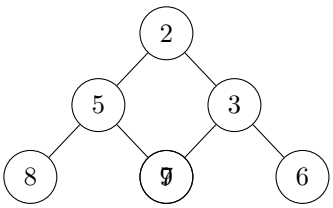
3.1 Min Heap

In a **min heap**, every parent is less than or equal to each of its children. This guarantees that the **smallest value is at the root**, but it does *not* mean the entire array is sorted.

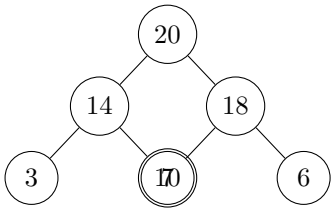
3.2 Max Heap

In a **max heap**, every parent is greater than or equal to each of its children. This guarantees that the **largest value is at the root**.

Min heap



Max heap



Remember: Siblings do not have to be ordered relative to each other. A min heap only compares each parent with its children, unlike a BST, which compares values across entire left and right subtrees.

4 Heap Operations

4.1 Insertion and Bubble Up

When we insert a new value, we always put it in the next open position at the bottom (the end of the array). This preserves completeness, but the new value might be smaller than its parent in a min heap. We then **bubble up**: compare with the parent and swap while the heap-order rule is violated.

Let's insert 3 into the min heap $[4, 9, 7, 15, 12, 10, 20]$.

4	9	7	15	12	10	20	3	Append 3
4	9	7	3	12	10	20	15	Swap with 15
4	3	7	9	12	10	20	15	Swap with 9
3	4	7	9	12	10	20	15	Swap with 4

In the worst case, the new value travels from a leaf to the root. A complete tree with n nodes has height $O(\log n)$, so insertion is $O(\log n)$.

```
void add(int value) {
    heap.add(value);
    int i = heap.size() - 1;
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (heap.get(parent) <= heap.get(i)) break;
        swap(i, parent);
        i = parent;
    }
}
```

4.2 Removing the Root and Bubble Down

The standard heap removal operation removes the root, because that is the minimum or maximum. We replace the root with the last element, delete the last array position, then **bubble down** until the heap-order property is restored.

Starting with $[3, 4, 7, 9, 12, 10, 20, 15]$, remove the minimum 3:

15	4	7	9	12	10	20	Move last element to root
4	15	7	9	12	10	20	Swap with smaller child (4)
4	9	7	15	12	10	20	Swap with smaller child (9)

Remember: When bubbling down in a min heap, compare both children and swap with the *smaller* one. Swapping with the wrong child may leave the heap invalid.

```
int removeMin() {
    if (heap.isEmpty()) throw new NoSuchElementException();
    int answer = heap.get(0);
    int last = heap.remove(heap.size() - 1);
    if (heap.isEmpty()) return answer;
    heap.set(0, last);
    int i = 0;
    while (true) {
        int left = 2 * i + 1, right = 2 * i + 2;
        if (left >= heap.size()) break;
        int smaller = left;
        if (right < heap.size()
                && heap.get(right) < heap.get(left)) {
            smaller = right;
        }
        if (heap.get(i) <= heap.get(smaller)) break;
        swap(i, smaller);
        i = smaller;
    }
    return answer;
}
```

4.3 Peek

`peek()` returns the root without removing it. Since the root is at array index 0, it takes $O(1)$ time.

```
int peek() {
    if (heap.isEmpty()) throw new NoSuchElementException();
    return heap.get(0);
}
```

5 Building a Heap from an Unordered Array

Suppose we already have n values in an array. One option is to insert them one at a time, costing $O(n \log n)$ in the worst case. But there is a faster **bottom-up build heap** algorithm:

1. Start at the last non-leaf node, index $\lfloor n/2 \rfloor - 1$.
2. Bubble down that node until its subtree obeys the heap rule.

3. Move backward through all remaining internal nodes, ending at the root.

```
void buildHeap() {
    for (int i = heap.size() / 2 - 1; i >= 0; i--) {
        siftDown(i);
    }
}
```

Why is this $O(n)$ rather than $O(n \log n)$? Most nodes are near the leaves and can move only a tiny distance. Only a few nodes near the root can travel far. Summing the work across levels gives $O(n)$ total.

6 Complete Java Min Heap Implementation

The following implementation stores integer values in an `ArrayList<Integer>`. It includes insertion, peek, root removal, heap construction, and a helper for swapping elements. This is intentionally written without Java's built-in `PriorityQueue` so we can see how the data structure actually works.

```
import java.util.*;

public class MinHeap {
    private final ArrayList<Integer> heap = new ArrayList<>();

    public MinHeap() { }

    public MinHeap(int[] values) {
        for (int value : values) heap.add(value);
        for (int i = heap.size() / 2 - 1; i >= 0; i--) {
            siftDown(i);
        }
    }

    public int size() { return heap.size(); }
    public boolean isEmpty() { return heap.isEmpty(); }

    public int peek() {
        if (isEmpty()) throw new NoSuchElementException();
        return heap.get(0);
    }

    public void add(int value) {
        heap.add(value);
        int i = heap.size() - 1;
        while (i > 0) {
            int parent = (i - 1) / 2;
            if (heap.get(parent) <= heap.get(i)) break;
            swap(i, parent);
            i = parent;
        }
    }

    public int removeMin() {
        if (isEmpty()) throw new NoSuchElementException();
        int result = heap.get(0);
        int last = heap.remove(heap.size() - 1);
        if (!isEmpty()) {
            heap.set(0, last);
            siftDown(0);
        }
        return result;
```

```
    }

    private void siftDown(int i) {
        while (true) {
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            if (left >= heap.size()) return;
            int smaller = left;
            if (right < heap.size()
                    && heap.get(right) < heap.get(left)) {
                smaller = right;
            }
            if (heap.get(i) <= heap.get(smaller)) return;
            swap(i, smaller);
            i = smaller;
        }
    }

    private void swap(int i, int j) {
        int temp = heap.get(i);
        heap.set(i, heap.get(j));
        heap.set(j, temp);
    }

    @Override public String toString() {
        return heap.toString();
    }

    public static void main(String[] args) {
        MinHeap h = new MinHeap(new int[]{9, 4, 7, 1, 6});
        System.out.println(h);       // heap order, not sorted order
        h.add(2);
        System.out.println(h.peek()); // 1
        while (!h.isEmpty()) {
            System.out.print(h.removeMin() + " ");
        }
        // 1 2 4 6 7 9
    }
}
```

7 PriorityQueue in Java

Java provides `java.util.PriorityQueue`, normally implemented with a min heap. By default, the smallest element according to the natural ordering is removed first. We can also pass a `Comparator` to create max-heap behavior.

7.1 Min Heap with PriorityQueue

```
import java.util.*;
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(12);
pq.offer(4);
pq.offer(8);
System.out.println(pq.peek()); // 4
System.out.println(pq.poll()); // 4
System.out.println(pq.poll()); // 8
```

7.2 Max Heap with PriorityQueue

```
PriorityQueue<Integer> maxHeap =
        new PriorityQueue<>(Comparator.reverseOrder());
maxHeap.offer(12);
maxHeap.offer(4);
maxHeap.offer(8);
System.out.println(maxHeap.poll()); // 12
```

Remember: Printing a `PriorityQueue` or iterating through it does *not* guarantee sorted order. The priority ordering is guaranteed for the head returned by `peek()` and `poll()`.

7.3 Priority Queues with Custom Objects

Suppose we have tasks with priority numbers. Smaller priority numbers represent more urgent tasks.

```
import java.util.*;
class Task {
    String name;
    int priority;
    Task(String name, int priority) {
        this.name = name;
        this.priority = priority;
    }
}
public class TaskDemo {
    public static void main(String[] args) {
        PriorityQueue<Task> tasks = new PriorityQueue<>(
            Comparator.comparingInt(t -> t.priority));
        tasks.offer(new Task("Email", 3));
        tasks.offer(new Task("Emergency", 1));
        tasks.offer(new Task("Homework", 2));
        while (!tasks.isEmpty()) {
            System.out.println(tasks.poll().name);
        }
        // Emergency, Homework, Email
    }
}
```

8 Heaps vs Binary Search Trees

Although both are binary trees, their ordering rules serve different purposes. A BST guarantees that all values in a node's left subtree are smaller (under the chosen duplicate policy), and values in the right subtree are larger. A heap only compares each parent with its children. Therefore a BST supports efficient arbitrary search when balanced, whereas a heap excels at repeatedly finding the minimum or maximum.

Feature	Binary Heap	Balanced BST
Main ordering	Parent vs children	Left vs node vs right
Minimum/maximum	$O(1)$ for designated extreme	$O(\log n)$ generally
Search for arbitrary value	$O(n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Remove designated extreme	$O(\log n)$	$O(\log n)$
Typical storage	Array	Linked nodes

Remember: An ordinary unbalanced BST can degrade to $O(n)$ search and update time. The balanced BST column assumes a self-balancing tree or a tree whose height remains logarithmic.

9 Time Complexity Summary

Operation	Binary min/max heap	Explanation
Peek root	$O(1)$	Read index 0
Insert	$O(\log n)$	Bubble up one tree height
Remove root	$O(\log n)$	Bubble down one tree height
Search arbitrary value	$O(n)$	No global sorted order
Build heap from array	$O(n)$	Bottom-up heapify
Space	$O(n)$	Store all values

10 Heap Sort

Heap sort uses a heap to arrange elements in sorted order. For ascending order, a max heap is typically used. First build a max heap, then repeatedly swap its root (the largest remaining element) with the last element of the unsorted region, shrink the region, and restore the max heap property.

Heap sort takes $O(n \log n)$ time in the best, average, and worst cases, and can be implemented in place with $O(1)$ auxiliary array space. It is *not stable* in its usual form.

11 Worked Practice Questions

1. **What are the child indices of index 3?** Left: $2(3) + 1 = 7$. Right: $2(3) + 2 = 8$.
2. **What is the parent of index 6?** $\lfloor (6 - 1)/2 \rfloor = 2$.
3. **Is** $[1, 4, 2, 8, 7, 3]$ **a valid min heap?** Yes. Each parent is less than or equal to its existing children.
4. **Is** $[1, 5, 2, 3]$ **a valid min heap?** No. The value 5 at index 1 has child 3 at index 3, violating the min-heap rule.
5. **Why isn't searching a heap** $O(\log n)$**?** There is no rule saying an arbitrary target must be in a specific subtree. In the worst case, we inspect all nodes.
6. **Can we remove an arbitrary heap element?** It is possible if we know its index and repair the heap afterward, but the *standard* priority queue operation removes the root. Java's `PriorityQueue.remove(Object)` can remove a matching element, typically requiring $O(n)$ to locate it.
7. **Why is build heap** $O(n)$**?** Most nodes are near the leaves and need very little bubbling; total work across all levels sums to linear time.

12 Common Mistakes to Avoid

- Confusing a heap with a BST. Heap order is local, not a full sorted-tree ordering.
- Using $i/2$ as the parent index for a zero-indexed Java array. Use $(i - 1)/2$.
- Forgetting to keep the tree complete when inserting.

- Swapping with the wrong child during bubble down.
- Assuming `PriorityQueue` iteration is sorted.
- Assuming every heap operation is $O(\log n)$; `peek()` is $O(1)$ and arbitrary search is $O(n)$.